

Assignment3

**Part A: Creating a DataFrame **

```
import numpy as np
import pandas as pd

# Initialize the NumPy random number generator using seed value 10
np.random.seed(10)

print("NumPy and Pandas imported. Random seed initialized to 10.")
```

NumPy and Pandas imported. Random seed initialized to 10.

```
# 2. Create a synthetic student dataset

# Define the number of students
num_students = 1000

print(f"Creating a synthetic dataset for {num_students} students...")

# (a) ID: Unique IDs from 1 to num_students
student_ids = np.arange(1, num_students + 1)

# (b) Age: Random integers between 18 and 25
student_age = np.random.randint(18, 26, size=num_students)

# (c) Height: Random floats between 150 and 190 cm
student_height = np.random.uniform(150, 190, size=num_students).round(2)

# (d) Weight: Random floats between 50 and 90 kg
student_weight = np.random.uniform(50, 90, size=num_students).round(2)

# (e) Gender: Randomly choose 'Male' or 'Female'
genders = np.random.choice(['Male', 'Female'], size=num_students)

# (f) Department: Randomly choose from a list of departments
departments = np.random.choice(['CS', 'EE', 'ME', 'CE', 'EC', 'IT'], size=num_students)

# (g) CGPA: Random floats between 6.0 and 10.0
cgpas = np.random.uniform(6.0, 10.0, size=num_students).round(2)

# (h) Semester: Random integers between 1 and 8
semesters = np.random.randint(1, 9, size=num_students)

# (i) Hosteller: Randomly choose 'Yes' or 'No'
hostellers = np.random.choice(['Yes', 'No'], size=num_students)

# (j) Grade: Assign based on CGPA
def assign_grade(cgpa):
    if cgpa >= 9.0:
        return 'A'
    elif cgpa >= 8.0:
        return 'B'
    elif cgpa >= 7.0:
        return 'C'
    elif cgpa >= 6.0:
        return 'D'
```

```
        else:
            return 'F'

grades = np.vectorize(assign_grade)(cgpas)

# Create a Pandas DataFrame
student_data = pd.DataFrame({
    'ID': student_ids,
    'Age': student_age,
    'Height': student_height,
    'Weight': student_weight,
    'Gender': genders,
    'Department': departments,
    'CGPA': cgpas,
    'Semester': semesters,
    'Hosteller': hostellers,
    'Grade': grades
})

print("\nSynthetic student dataset created successfully:")
display(student_data.head()) # Display the first 5 rows

print(f"\nDataset shape: {student_data.shape}")
print("Dataset info:")
display(student_data.info())
```

Creating a synthetic dataset for 1000 students...

Synthetic student dataset created successfully:

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No

Dataset shape: (1000, 10)

Dataset info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	ID	1000 non-null	int64
1	Age	1000 non-null	int64
2	Height	1000 non-null	float64
3	Weight	1000 non-null	float64
4	Gender	1000 non-null	object
5	Department	1000 non-null	object
6	CGPA	1000 non-null	float64
7	Semester	1000 non-null	int64
8	Hosteller	1000 non-null	object
9	Grade	1000 non-null	object

dtypes: float64(3), int64(3), object(4)

memory usage: 78.3+ KB

None

```
# 3. Store the DataFrame data in a Python dictionary named 'students'
# Using 'records' format, where each row becomes a dictionary in a list.
students = student_data.to_dict(orient='records')
```

```
print("DataFrame converted to a list of dictionaries named 'students'.")
print("Displaying the first 5 entries of the 'students' dictionary:")
display(students[:5])
```

```
print(f"\nTotal number of student records: {len(students)}")
```

DataFrame converted to a list of dictionaries named 'students'.
Displaying the first 5 entries of the 'students' dictionary:

```
[{'ID': 1,
  'Age': 19,
  'Height': 151.19,
  'Weight': 70.54,
  'Gender': 'Female',
  'Department': 'IT',
  'CGPA': 8.81,
  'Semester': 6,
  'Hosteller': 'Yes',
  'Grade': 'B'},
 {'ID': 2,
  'Age': 23,
  'Height': 150.46,
  'Weight': 80.77,
  'Gender': 'Female',
  'Department': 'EC',
  'CGPA': 6.43,
  'Semester': 8,
  'Hosteller': 'No',
  'Grade': 'D'},
 {'ID': 3,
  'Age': 22,
  'Height': 163.27,
  'Weight': 58.03,
  'Gender': 'Female',
  'Department': 'EE',
  'CGPA': 8.67,
  'Semester': 8,
  'Hosteller': 'Yes',
  'Grade': 'B'},
 {'ID': 4,
  'Age': 25,
  'Height': 152.46,
  'Weight': 70.33,
  'Gender': 'Male',
  'Department': 'CE',
  'CGPA': 8.86,
  'Semester': 1,
  'Hosteller': 'Yes',
  'Grade': 'B'},
 {'ID': 5,
  'Age': 18,
  'Height': 173.93,
  'Weight': 85.3,
  'Gender': 'Female',
  'Department': 'IT',
  'CGPA': 9.13,
  'Semester': 6,
  'Hosteller': 'No',
  'Grade': 'A'}]
```

Total number of student records: 1000

```
# 4. Create a Pandas DataFrame named 'df' from the 'students' dictionary (li
df = pd.DataFrame(students)
```

```
print("DataFrame 'df' created from 'students' list of dictionaries.")
print("Displaying the first 5 rows of df:")
display(df.head())

print(f"\nDataset shape: {df.shape}")
print("Dataset info:")
display(df.info())
```

DataFrame 'df' created from 'students' list of dictionaries.
Displaying the first 5 rows of df:

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No

Dataset shape: (1000, 10)

Dataset info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	ID	1000 non-null	int64
1	Age	1000 non-null	int64
2	Height	1000 non-null	float64
3	Weight	1000 non-null	float64
4	Gender	1000 non-null	object
5	Department	1000 non-null	object
6	CGPA	1000 non-null	float64
7	Semester	1000 non-null	int64
8	Hosteller	1000 non-null	object
9	Grade	1000 non-null	object

dtypes: float64(3), int64(3), object(4)

memory usage: 78.3+ KB

None

```
import numpy as np
import pandas as pd

# Initialize the NumPy random number generator using seed value 10
np.random.seed(10)

print("NumPy and Pandas imported. Random seed initialized to 10.")
```

NumPy and Pandas imported. Random seed initialized to 10.

2. Create a synthetic student dataset

```
# Define the number of students
num_students = 1000
```

```

print(f"Creating a synthetic dataset for {num_students} students...")

# (a) ID: Unique IDs from 1 to num_students
student_ids = np.arange(1, num_students + 1)

# (b) Age: Random integers between 18 and 25
student_age = np.random.randint(18, 26, size=num_students)

# (c) Height: Random floats between 150 and 190 cm
student_height = np.random.uniform(150, 190, size=num_students).round(2)

# (d) Weight: Random floats between 50 and 90 kg
student_weight = np.random.uniform(50, 90, size=num_students).round(2)

# (e) Gender: Randomly choose 'Male' or 'Female'
genders = np.random.choice(['Male', 'Female'], size=num_students)

# (f) Department: Randomly choose from a list of departments
departments = np.random.choice(['CS', 'EE', 'ME', 'CE', 'EC', 'IT'], size=num_students)

# (g) CGPA: Random floats between 6.0 and 10.0
cgpas = np.random.uniform(6.0, 10.0, size=num_students).round(2)

# (h) Semester: Random integers between 1 and 8
semesters = np.random.randint(1, 9, size=num_students)

# (i) Hosteller: Randomly choose 'Yes' or 'No'
hostellers = np.random.choice(['Yes', 'No'], size=num_students)

# (j) Grade: Assign based on CGPA
def assign_grade(cgpa):
    if cgpa >= 9.0:
        return 'A'
    elif cgpa >= 8.0:
        return 'B'
    elif cgpa >= 7.0:
        return 'C'
    elif cgpa >= 6.0:
        return 'D'
    else:
        return 'F'

grades = np.vectorize(assign_grade)(cgpas)

# Create a Pandas DataFrame
student_data = pd.DataFrame({
    'ID': student_ids,
    'Age': student_age,
    'Height': student_height,
    'Weight': student_weight,
    'Gender': genders,
    'Department': departments,
    'CGPA': cgpas,
    'Semester': semesters,
    'Hosteller': hostellers,
    'Grade': grades
})

print("\nSynthetic student dataset created successfully:")
display(student_data.head()) # Display the first 5 rows

print(f"\nDataset shape: {student_data.shape}")

```

```
print("Dataset info:")
display(student_data.info())
```

Creating a synthetic dataset for 1000 students...

Synthetic student dataset created successfully:

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No

Dataset shape: (1000, 10)

Dataset info:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 1000 entries, 0 to 999

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	ID	1000 non-null	int64
1	Age	1000 non-null	int64
2	Height	1000 non-null	float64
3	Weight	1000 non-null	float64
4	Gender	1000 non-null	object
5	Department	1000 non-null	object
6	CGPA	1000 non-null	float64
7	Semester	1000 non-null	int64
8	Hosteller	1000 non-null	object
9	Grade	1000 non-null	object

dtypes: float64(3), int64(3), object(4)

memory usage: 78.3+ KB

None

```
# 3. Store the DataFrame data in a Python dictionary named 'students'
# Using 'records' format, where each row becomes a dictionary in a list.
students = student_data.to_dict(orient='records')
```

```
print("DataFrame converted to a list of dictionaries named 'students'.")
print("Displaying the first 5 entries of the 'students' dictionary:")
display(students[:5])
```

```
print(f"\nTotal number of student records: {len(students)}")
```

DataFrame converted to a list of dictionaries named 'students'.
Displaying the first 5 entries of the 'students' dictionary:

```
[{'ID': 1,
  'Age': 19,
  'Height': 151.19,
  'Weight': 70.54,
  'Gender': 'Female',
  'Department': 'IT',
  'CGPA': 8.81,
  'Semester': 6,
  'Hosteller': 'Yes',
  'Grade': 'B'},
 {'ID': 2,
  'Age': 23,
  'Height': 150.46,
  'Weight': 80.77,
  'Gender': 'Female',
  'Department': 'EC',
  'CGPA': 6.43,
  'Semester': 8,
  'Hosteller': 'No',
  'Grade': 'D'},
 {'ID': 3,
  'Age': 22,
  'Height': 163.27,
  'Weight': 58.03,
  'Gender': 'Female',
  'Department': 'EE',
  'CGPA': 8.67,
  'Semester': 8,
  'Hosteller': 'Yes',
  'Grade': 'B'},
 {'ID': 4,
  'Age': 25,
  'Height': 152.46,
  'Weight': 70.33,
  'Gender': 'Male',
  'Department': 'CE',
  'CGPA': 8.86,
  'Semester': 1,
  'Hosteller': 'Yes',
  'Grade': 'B'},
 {'ID': 5,
  'Age': 18,
  'Height': 173.93,
  'Weight': 85.3,
  'Gender': 'Female',
  'Department': 'IT',
  'CGPA': 9.13,
  'Semester': 6,
  'Hosteller': 'No',
  'Grade': 'A'}]
```

Total number of student records: 1000

4. Create a Pandas DataFrame named 'df' from the 'students' dictionary (li
df = pd.DataFrame(students)

```

print("DataFrame 'df' created from 'students' list of dictionaries.")
print("Displaying the first 5 rows of df:")
display(df.head())

print(f"\nDataset shape: {df.shape}")
print("Dataset info:")
display(df.info())

```

DataFrame 'df' created from 'students' list of dictionaries.
Displaying the first 5 rows of df:

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No

Dataset shape: (1000, 10)

Dataset info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	ID	1000 non-null	int64
1	Age	1000 non-null	int64
2	Height	1000 non-null	float64
3	Weight	1000 non-null	float64
4	Gender	1000 non-null	object
5	Department	1000 non-null	object
6	CGPA	1000 non-null	float64
7	Semester	1000 non-null	int64
8	Hosteller	1000 non-null	object
9	Grade	1000 non-null	object

dtypes: float64(3), int64(3), object(4)

memory usage: 78.3+ KB

None

```

print("\n--- Displaying the 'Age' column ---")
display(df['Age'])

```

--- Displaying the 'Age' column ---

	Age
0	19
1	23
2	22
3	25
4	18
...	...
995	18
996	19
997	21
998	21
999	25

1000 rows × 1 columns

dtype: int64

```
print("\n--- First 5 rows of the DataFrame (df.head()) ---")
display(df.head())
```

```
print("\n--- Last 5 rows of the DataFrame (df.tail()) ---")
display(df.tail())
```

```
--- First 5 rows of the DataFrame (df.head()) ---
```

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No

```
--- Last 5 rows of the DataFrame (df.tail()) ---
```

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
995	996	18	174.97	66.92	Male	ME	9.89	7	
996	997	19	162.06	84.10	Male	CS	6.61	4	
997	998	21	161.62	75.22	Female	CE	9.69	4	
998	999	21	169.72	74.11	Male	IT	6.43	5	
999	1000	25	175.82	52.40	Female	CE	7.59	6	

Part B: Exploring the DataFrame

```
print("\n--- First 7 rows of the DataFrame ---")  
display(df.head(7))
```

```
--- First 7 rows of the DataFrame ---
```

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No
5	6	19	185.44	69.65	Female	EC	8.57	2	Yes
6	7	21	166.49	88.16	Male	CS	8.27	1	Yes

```
print("\n--- Last 3 rows of the DataFrame ---")  
display(df.tail(3))
```

```
--- Last 3 rows of the DataFrame ---
```

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
997	998	21	161.62	75.22	Female	CE	9.69	4	
998	999	21	169.72	74.11	Male	IT	6.43	5	
999	1000	25	175.82	52.40	Female	CE	7.59	6	

```
print("\n--- Shape of the DataFrame ---")
print(df.shape)
```

```
--- Shape of the DataFrame ---
(1000, 10)
```

```
print("\n--- Column Names of the DataFrame ---")
print(df.columns.tolist())
```

```
--- Column Names of the DataFrame ---
['ID', 'Age', 'Height', 'Weight', 'Gender', 'Department', 'CGPA', 'Semester', 'Hosteller', 'Grade']
```

```
print("\n--- Data Types of Each Column ---")
display(df.dtypes)
```

```
--- Data Types of Each Column ---
```

	0
ID	int64
Age	int64
Height	float64
Weight	float64
Gender	object
Department	object
CGPA	float64
Semester	int64
Hosteller	object
Grade	object

dtype: object

```
print("\n--- Summary Information of the DataFrame ---")
```

```
df.info()
```

```
--- Summary Information of the DataFrame ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  -
0   ID           1000 non-null   int64
1   Age          1000 non-null   int64
2   Height       1000 non-null   float64
3   Weight       1000 non-null   float64
4   Gender       1000 non-null   object
5   Department   1000 non-null   object
6   CGPA         1000 non-null   float64
7   Semester     1000 non-null   int64
8   Hosteller    1000 non-null   object
9   Grade        1000 non-null   object
dtypes: float64(3), int64(3), object(4)
memory usage: 78.3+ KB
```

```
print("\n--- Descriptive Statistics for Numerical Columns ---")
display(df.describe())
```

```
--- Descriptive Statistics for Numerical Columns ---
```

	ID	Age	Height	Weight	CGPA	Semester
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	500.500000	21.399000	169.944530	69.508180	7.963180	4.521000
std	288.819436	2.365882	11.755481	11.683919	1.114398	2.281000
min	1.000000	18.000000	150.010000	50.100000	6.000000	1.000000
25%	250.750000	19.000000	159.677500	59.192500	7.050000	3.000000
50%	500.500000	21.000000	169.370000	69.665000	8.000000	5.000000
75%	750.250000	24.000000	180.440000	79.705000	8.860000	7.000000
max	1000.000000	25.000000	189.950000	90.000000	10.000000	8.000000

Part C: Reading Data

```
print("\n--- Displaying the 'Age' column ---")
display(df['Age'])
```

```
--- Displaying the 'Age' column ---
```

	Age
--	-----

0	19
---	----

1	23
---	----

2	22
---	----

3	25
---	----

4	18
---	----

...	...
-----	-----

995	18
-----	----

996	19
-----	----

997	21
-----	----

998	21
-----	----

999	25
-----	----

1000 rows × 1 columns

dtype: int64

```
print("\n--- Displaying 'Age' and 'CGPA' columns ---")
display(df[['Age', 'CGPA']])
```

```
--- Displaying 'Age' and 'CGPA' columns ---
```

	Age	CGPA
0	19	8.81
1	23	6.43
2	22	8.67
3	25	8.86
4	18	9.13
...	...	...
995	18	9.89
996	19	6.61
997	21	9.69
998	21	6.43
999	25	7.59

1000 rows × 2 columns

```
print("\n--- Displaying Rows 2 to 6 (inclusive) ---")
display(df.iloc[2:7])
```

```
--- Displaying Rows 2 to 6 (inclusive) ---
```

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No
5	6	19	185.44	69.65	Female	EC	8.57	2	Yes
6	7	21	166.49	88.16	Male	CS	8.27	1	Yes

```
print("\n--- Displaying Rows 2 to 6 with 'Age', 'Department', and 'CGPA' columns ---")
display(df.iloc[2:7][['Age', 'Department', 'CGPA']])
```

```
--- Displaying Rows 2 to 6 with 'Age', 'Department', and 'CGPA' columns
```

	Age	Department	CGPA
2	22	EE	8.67
3	25	CE	8.86
4	18	IT	9.13
5	19	EC	8.57
6	21	CS	8.27

```
print("\n--- Displaying rows 1, 5, 8, and 10 with 'Height' and 'Weight' columns")
display(df.iloc[[0, 4, 7, 9]][['Height', 'Weight']])
```

```
--- Displaying rows 1, 5, 8, and 10 with 'Height' and 'Weight' columns
```

	Height	Weight
0	151.19	70.54
4	173.93	85.30
7	151.53	75.75
9	153.28	59.50

```
print("\n--- Students with CGPA greater than 8 (ID, Department, CGPA) ---")
display(df[df['CGPA'] > 8][['ID', 'Department', 'CGPA']])
```

--- Students with CGPA greater than 8 (ID, Department, CGPA) ---

ID Department CGPA			
0	1	IT	8.81
2	3	EE	8.67
3	4	CE	8.86
4	5	IT	9.13
5	6	EC	8.57
...	...	...	...
989	990	CS	8.73
992	993	EE	8.89
993	994	EE	9.35
995	996	ME	9.89
997	998	CE	9.69

498 rows × 3 columns

```
print("\n--- Student with ID 5 ---")
display(df[df['ID'] == 5])
```

--- Student with ID 5 ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
4	5	18	173.93	85.3	Female	IT	9.13	6	No

```
print("\n--- Students from CS Department ---")
display(df[df['Department'] == 'CS'])
```

--- Students from CS Department ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hostel
6	7	21	166.49	88.16	Male	CS	8.27	1	
8	9	19	153.23	52.34	Male	CS	6.40	3	
21	22	21	183.66	82.78	Female	CS	8.73	4	
36	37	21	173.29	68.38	Female	CS	6.42	1	
39	40	23	188.68	57.77	Male	CS	7.43	6	
...	...	...	...	...	...	...	...	...	
976	977	25	165.62	56.50	Female	CS	6.52	7	
977	978	21	161.43	71.78	Male	CS	8.34	8	
986	987	24	158.44	88.73	Female	CS	9.48	1	
989	990	24	174.01	77.36	Female	CS	8.73	4	
996	997	19	162.06	84.10	Male	CS	6.61	4	

165 rows × 10 columns

```
print("\n--- Students with CGPA greater than 9 ---")
display(df[df['CGPA'] > 9])
```

--- Students with CGPA greater than 9 ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hostel
4	5	18	173.93	85.30	Female	IT	9.13	6	
7	8	22	151.53	75.75	Female	ME	9.87	3	
13	14	20	184.45	59.40	Male	EC	9.05	6	
16	17	18	161.48	67.43	Male	IT	9.60	7	
18	19	18	174.68	88.31	Female	IT	9.33	7	
...	...	...	...	...	...	...	...	...	
986	987	24	158.44	88.73	Female	CS	9.48	1	
988	989	19	167.92	86.94	Female	ME	9.64	2	
993	994	24	161.68	64.46	Male	EE	9.35	3	
995	996	18	174.97	66.92	Male	ME	9.89	7	
997	998	21	161.62	75.22	Female	CE	9.69	4	

218 rows × 10 columns

```
print("\n--- First three rows using integer indexing (iloc[0:3]) ---")
display(df.iloc[0:3])
```

```
--- First three rows using integer indexing (iloc[0:3]) ---
```

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes

Part D: Creating New Data

```
newStudent = {
    'ID': 21,
    'Age': 20,
    'Height': 172.5,
    'Weight': 68.4,
    'Gender': 'Male',
    'Department': 'CSE',
    'CGPA': 8.75,
    'Semester': 5,
    'Hosteller': 'Yes',
    'Grade': 'A'
}
```

```
# Convert the new student dictionary to a DataFrame row
new_student_df = pd.DataFrame([newStudent])
```

```
# Append the new student to the existing DataFrame
df = pd.concat([df, new_student_df], ignore_index=True)
```

```
print("\n--- New student added to DataFrame. Displaying the last 5 rows of the DataFrame ---")
display(df.tail())
```

```
--- New student added to DataFrame. Displaying the last 5 rows of the DataFrame ---
```

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
996	997	19	162.06	84.10	Male	CS	6.61	4	No
997	998	21	161.62	75.22	Female	CE	9.69	4	No
998	999	21	169.72	74.11	Male	IT	6.43	5	No
999	1000	25	175.82	52.40	Female	CE	7.59	6	No
1000	21	20	172.50	68.40	Male	CSE	8.75	5	Yes

```
# Create a new column named BMI
df['BMI'] = df['Weight'] / ((df['Height'] / 100)**2)
```

```
print("\n--- DataFrame with new 'BMI' column (first 5 rows) ---")
display(df.head())
```

--- DataFrame with new 'BMI' column (first 5 rows) ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No

```
# Create a new boolean column named 'Passed'
# Assuming a student passes if their CGPA is 6.0 or greater
df['Passed'] = df['CGPA'] >= 6.0
```

```
print("\n--- DataFrame with new 'Passed' column (first 5 rows) ---")
display(df.head())
```

--- DataFrame with new 'Passed' column (first 5 rows) ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No

```
df['BonusCGPA'] = df['CGPA'] + 0.25
```

```
print("\n--- DataFrame with new 'BonusCGPA' column (first 5 rows) ---")
display(df.head())
```

--- DataFrame with new 'BonusCGPA' column (first 5 rows) ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No

```
conditions = [  
    df['CGPA'] >= 9,  
    (df['CGPA'] >= 8) & (df['CGPA'] < 9),  
    (df['CGPA'] >= 7) & (df['CGPA'] < 8),  
    df['CGPA'] < 7  
]  
  
choices = ['Excellent', 'Good', 'Average', 'Poor']  
  
df['Performance'] = np.select(conditions, choices, default='Unknown')  
  
print("\n--- DataFrame with new 'Performance' column (first 5 rows) ---")  
display(df.head())
```

--- DataFrame with new 'Performance' column (first 5 rows) ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	Yes
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	Yes
3	4	25	152.46	70.33	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No

```
# (a) Set the CGPA of the student whose ID is 5 to NaN.  
df.loc[df['ID'] == 5, 'CGPA'] = np.nan  
  
# (b) Set the Height of the student whose ID is 12 to NaN.  
df.loc[df['ID'] == 12, 'Height'] = np.nan  
  
# (c) Set the Department of the student whose ID is 18 to NaN.  
df.loc[df['ID'] == 18, 'Department'] = np.nan  
  
print("--- DataFrame entries after introducing NaNs ---")  
print("Student with ID 5 (CGPA should be NaN):")  
display(df[df['ID'] == 5])  
  
print("\nStudent with ID 12 (Height should be NaN):")  
display(df[df['ID'] == 12])
```

```
print("\nStudent with ID 18 (Department should be NaN):")
display(df[df['ID'] == 18])
```

--- DataFrame entries after introducing NaNs ---
 Student with ID 5 (CGPA should be NaN):

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
4	5	18	173.93	85.3	Female	IT	NaN	6	No

Student with ID 12 (Height should be NaN):

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
11	12	23	NaN	70.61	Female	CE	6.63	2	Yes

Student with ID 18 (Department should be NaN):

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
17	18	20	183.72	78.16	Female	NaN	8.01	1	Yes

```
print("--- Updated DataFrame after introducing NaNs ---")
display(df)
```

--- Updated DataFrame after introducing NaNs ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	8.81	6	No
1	2	23	150.46	80.77	Female	EC	6.43	8	No
2	3	22	163.27	58.03	Female	EE	8.67	8	No
3	4	25	152.46	70.33	Male	CE	8.86	1	No
4	5	18	173.93	85.30	Female	IT	NaN	6	No
...	...	...	...	...	...	...	...	...	...
996	997	19	162.06	84.10	Male	CS	6.61	4	No
997	998	21	161.62	75.22	Female	CE	9.69	4	No
998	999	21	169.72	74.11	Male	IT	6.43	5	No
999	1000	25	175.82	52.40	Female	CE	7.59	6	No
1000	21	20	172.50	68.40	Male	CSE	8.75	5	No

1001 rows × 14 columns

Part E: Updating Data

```
# Update the CGPA of the student whose ID is 3 to 9.75
df.loc[df['ID'] == 3, 'CGPA'] = 9.75
```

```
print("--- Updated CGPA for student with ID 3 ---")
display(df[df['ID'] == 3])
```

--- Updated CGPA for student with ID 3 ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
2	3	22	163.27	58.03	Female	EE	9.75	8	Yes

```
df.loc[df['ID'] == 8, 'Department'] = 'CSE'
```

```
print("--- Updated Department for student with ID 8 ---")
display(df[df['ID'] == 8])
```

--- Updated Department for student with ID 8 ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
7	8	22	151.53	75.75	Female	CSE	9.87	3	No

```
df['CGPA'] += 0.25
```

```
print("--- CGPA of all students increased by 0.25 ---")
display(df.head())
```

--- CGPA of all students increased by 0.25 ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	70.54	Female	IT	9.06	6	Yes
1	2	23	150.46	80.77	Female	EC	6.68	8	No
2	3	22	163.27	58.03	Female	EE	10.00	8	Yes
3	4	25	152.46	70.33	Male	CE	9.11	1	Yes
4	5	18	173.93	85.30	Female	IT	NaN	6	No

```
df['Weight'] += 2
```

```
print("--- Weight of all students increased by 2 kg ---")
display(df.head())
```

--- Weight of all students increased by 2 kg ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	72.54	Female	IT	9.06	6	Yes
1	2	23	150.46	82.77	Female	EC	6.68	8	No
2	3	22	163.27	60.03	Female	EE	10.00	8	Yes
3	4	25	152.46	72.33	Male	CE	9.11	1	Yes
4	5	18	173.93	87.30	Female	IT	NaN	6	No

```
print("--- Grade distribution BEFORE change ---")
display(df['Grade'].value_counts().sort_index())

df.loc[df['Grade'] == 'C', 'Grade'] = 'B'

print("\n--- All 'Grade C' students have been changed to 'Grade B'. ---")
print("--- Grade distribution AFTER change ---")
display(df['Grade'].value_counts().sort_index())

print("\n--- First 10 rows of the updated DataFrame: ---")
display(df.head(10))
```

```
--- Grade distribution BEFORE change ---
```

```
count
```

```
Grade
```

A	221
B	281
C	263
D	236

```
dtype: int64
```

```
--- All 'Grade C' students have been changed to 'Grade B'. ---
```

```
--- Grade distribution AFTER change ---
```

```
count
```

```
Grade
```

A	221
B	544
D	236

```
dtype: int64
```

```
--- First 10 rows of the updated DataFrame: ---
```

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	72.54	Female	IT	9.06	6	Yes
1	2	23	150.46	82.77	Female	EC	6.68	8	No
2	3	22	163.27	60.03	Female	EE	10.00	8	Yes
3	4	25	152.46	72.33	Male	CE	9.11	1	Yes
4	5	18	173.93	87.30	Female	IT	NaN	6	No
5	6	19	185.44	71.65	Female	EC	8.82	2	Yes
6	7	21	166.49	90.16	Male	CS	8.52	1	Yes
7	8	22	151.53	77.75	Female	CSE	10.12	3	No
8	9	19	153.23	54.34	Male	CS	6.65	3	No
9	10	23	153.28	61.50	Male	CE	8.66	4	Yes

```
print("--- Fully Updated DataFrame ---")  
display(df)
```

--- Fully Updated DataFrame ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Host
0	1	19	151.19	72.54	Female	IT	9.06	6	
1	2	23	150.46	82.77	Female	EC	6.68	8	
2	3	22	163.27	60.03	Female	EE	10.00	8	
3	4	25	152.46	72.33	Male	CE	9.11	1	
4	5	18	173.93	87.30	Female	IT	NaN	6	
...	...	...	...	...	...	...	...	...	...
996	997	19	162.06	86.10	Male	CS	6.86	4	
997	998	21	161.62	77.22	Female	CE	9.94	4	
998	999	21	169.72	76.11	Male	IT	6.68	5	
999	1000	25	175.82	54.40	Female	CE	7.84	6	
1000	21	20	172.50	70.40	Male	CSE	9.00	5	

1001 rows × 14 columns

Part F: Deleting Data

```
print("\n--- Deleting student with ID 10 ---")
df = df[df['ID'] != 10]

print("Student with ID 10 deleted. Displaying the first 5 rows of the updated DataFrame")
display(df.head())
```

--- Deleting student with ID 10 ---

Student with ID 10 deleted. Displaying the first 5 rows of the updated DataFrame

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	72.54	Female	IT	9.06	6	Yes
1	2	23	150.46	82.77	Female	EC	6.68	8	No
2	3	22	163.27	60.03	Female	EE	10.00	8	Yes
3	4	25	152.46	72.33	Male	CE	9.11	1	Yes
4	5	18	173.93	87.30	Female	IT	NaN	6	No

```
print("\n--- Deleting the 'Grade' column ---")
df = df.drop(columns=['Grade'])

print("The 'Grade' column has been deleted. Displaying the first 5 rows of the updated DataFrame")
display(df.head())
```

--- Deleting the 'Grade' column ---

The 'Grade' column has been deleted. Displaying the first 5 rows of the

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	1	19	151.19	72.54	Female	IT	9.06	6	Yes
1	2	23	150.46	82.77	Female	EC	6.68	8	No
2	3	22	163.27	60.03	Female	EE	10.00	8	Yes
3	4	25	152.46	72.33	Male	CE	9.11	1	Yes
4	5	18	173.93	87.30	Female	IT	NaN	6	No

```
print("\n--- Deleting the first row ---")
df = df.drop(index=df.index[0])
```

```
print("The first row has been deleted. Displaying the first 5 rows of the up")
display(df.head())
```

--- Deleting the first row ---

The first row has been deleted. Displaying the first 5 rows of the upda

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
1	2	23	150.46	82.77	Female	EC	6.68	8	No
2	3	22	163.27	60.03	Female	EE	10.00	8	Yes
3	4	25	152.46	72.33	Male	CE	9.11	1	Yes
4	5	18	173.93	87.30	Female	IT	NaN	6	No
5	6	19	185.44	71.65	Female	EC	8.82	2	Yes

```
print("\n--- Deleting students with CGPA less than 7 ---")
df = df[df['CGPA'] >= 7]
```

```
print("Students with CGPA less than 7 deleted. Displaying the first 5 rows c")
display(df.head())
```

--- Deleting students with CGPA less than 7 ---

Students with CGPA less than 7 deleted. Displaying the first 5 rows of

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
2	3	22	163.27	60.03	Female	EE	10.00	8	Yes
3	4	25	152.46	72.33	Male	CE	9.11	1	Yes
5	6	19	185.44	71.65	Female	EC	8.82	2	Yes
6	7	21	166.49	90.16	Male	CS	8.52	1	Yes
7	8	22	151.53	77.75	Female	CSE	10.12	3	No

```
print("\n--- Resetting the DataFrame index ---")
df = df.reset_index(drop=True)
```

```
print("DataFrame index reset. Displaying the first 5 rows of the updated DataFrame")
display(df.head())
```

--- Resetting the DataFrame index ---

DataFrame index reset. Displaying the first 5 rows of the updated DataFrame

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller
0	3	22	163.27	60.03	Female	EE	10.00	8	Yes
1	4	25	152.46	72.33	Male	CE	9.11	1	Yes
2	6	19	185.44	71.65	Female	EC	8.82	2	Yes
3	7	21	166.49	90.16	Male	CS	8.52	1	Yes
4	8	22	151.53	77.75	Female	CSE	10.12	3	No

```
print("\n--- Displaying the fully updated DataFrame ---")
display(df)
```

--- Displaying the fully updated DataFrame ---

	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hostel
0	3	22	163.27	60.03	Female	EE	10.00	8	
1	4	25	152.46	72.33	Male	CE	9.11	1	
2	6	19	185.44	71.65	Female	EC	8.82	2	
3	7	21	166.49	90.16	Male	CS	8.52	1	
4	8	22	151.53	77.75	Female	CSE	10.12	3	
...	...	...	...	...	...	...	...	...	...
813	994	24	161.68	66.46	Male	EE	9.60	3	
814	996	18	174.97	68.92	Male	ME	10.14	7	
815	998	21	161.62	77.22	Female	CE	9.94	4	
816	1000	25	175.82	54.40	Female	CE	7.84	6	
817	21	20	172.50	70.40	Male	CSE	9.00	5	

818 rows × 13 columns

Part G: Filtering and Sorting

```
print("\n--- Students with Age greater than 20 ---")
display(df[df['Age'] > 20])
```

--- Students with Age greater than 20 ---

```
print("\n--- Female students belonging to the CSE department ---")
display(df[(df['Gender'] == 'Female') & (df['Department'] == 'CSE')])
```

```
---1 Female students belonging to the CSE department ---1
3 ID Age Height Weight Gender Male Department CGPA Semester Hosteller
4 8 22 151.53 77.75 Female CSE 10.12 3 No
12 20 24 155.62 83.79 Male EE 10.25 4
```

```
print("\n--- Hostellers with CGPA greater than 8 ---")
display(df[(df['Hosteller'] == 'Yes') & (df['CGPA'] > 8)])
```

```
810 990 24 174.01 79.36 Female CS 8.98 4
--- Hostellers with CGPA greater than 8 ---
813 994 24 161.68 66.46 Male EE 9.60 3
ID Age Height Weight Gender Department CGPA Semester Hostel
0 3 22 163.27 60.03 Female EE 10.00 8
816 1000 25 175.82 54.40 Female CE 7.84 6
1 4 25 152.46 72.33 Male CE 9.11 1
2 6 19 185.44 71.65 Female EC 8.82 2
3 7 21 166.49 90.16 Male CS 8.52 1
5 13 19 150.68 82.35 Male EE 8.84 6
... ... ... ... ... ... ...
808 988 22 169.97 69.68 Male IT 8.94 7
809 989 19 167.92 88.94 Female ME 9.89 2
812 993 19 183.85 72.50 Female EE 9.14 6
815 998 21 161.62 77.22 Female CE 9.94 4
817 21 20 172.50 70.40 Male CSE 9.00 5

272 rows x 13 columns
```

```
print("\n--- DataFrame sorted by CGPA in ascending order ---")
display(df.sort_values(by='CGPA', ascending=True))
```

```
--- DataFrame sorted by CGPA in ascending order ---
ID Age Height Weight Gender Department CGPA Semester Hostel
585 715 18 185.77 80.64 Male EE 7.00 1
689 840 23 175.72 58.82 Female CE 7.00 6
505 616 24 150.82 68.00 Female EC 7.00 1
```